

(unique) and commands that govern how the container should run. It represents the single unit of deployment within the Kubernetes ecosystem, a single instance of an application which might consist of one or many containers running with tight coupling and shared resources.

Pods in a Kubernetes cluster can be used in two main ways. The first is pods that run a single container. The ‘one-container-per-pod’ model is the most common Kubernetes use case. The second method involves pods that run multiple containers that need to work together.

A pod might encapsulate an application composed of multiple co-located containers that are tightly coupled and need to share resources.

ReplicaSet: The purpose of a ReplicaSet is to maintain a stable set of replica pods running at any given time. A ReplicaSet contains information about how many copies of a particular pod should be running. To create multiple pods to match the ReplicaSet criteria, Kubernetes uses the pod template. The link a ReplicaSet has to its pods is via the latter’s *metadata.ownerReferences* field, which specifies which resource owns the current object.

Services: Services are an abstraction to expose the functionality of a set of pods. With Kubernetes, you don’t need to modify your application to use an unfamiliar service discovery mechanism. Kubernetes gives pods their own IP addresses and a single DNS name for a set of pods, and can load-balance across them.

One major problem that services solve is the integration of the front-end and back-end of a Web application. Since Kubernetes provides IP addresses behind the scenes to pods, when the latter are killed and resurrected, the IP addresses are changed. This creates a big problem on the front-end side to connect a given back-end IP address to the corresponding front-end IP address. Services solve this problem by providing an abstraction over the pods — something akin to a load balancer.

Volumes: A Kubernetes volume has an explicit lifetime — the same as the pod that encloses it. Consequently, a volume outlives any container that runs within the pod and the data is preserved across container restarts. Of course, when a pod ceases to exist, the volume will cease to exist, too. Perhaps more important than this is that Kubernetes supports many types of volumes, and a pod can use any number of them simultaneously.

At its core, a volume is just a directory, possibly with some data in it, which is accessible to the containers in a pod. How that directory comes to be, the medium that backs it and its contents are determined by the particular volume type used.

Why Kubernetes?

Containers are a good way to bundle and run applications. In a production environment, you need to manage the containers that run the applications and ensure that there is no downtime. For example, if one container goes down, another needs to start. Wouldn’t it be nice if this could be automated by a system?

That’s where Kubernetes comes to the rescue! It provides a framework to run distributed systems resiliently. It takes care of scaling requirements, failover, deployment patterns, and more. For example, Kubernetes can easily manage a canary deployment for your system.

Kubernetes provides users with:

1. Service discovery and load balancing
2. Storage orchestration
3. Automated roll-outs and roll-backs
4. Automatic bin packing
5. Self-healing
6. Secret and configuration management

What can Kubernetes do?

In this section we will look at some code examples of how to use Kubernetes when building a Web application from scratch. We will create a simple back-end server using Flask in Python.

There are a few prerequisites for those who want to build a Web app from scratch. These are:

1. Basic understanding of Docker, Docker containers and Docker images. A quick refresher can be found at https://www.docker.com/sites/default/files/Docker_CheatSheet_08.09.2016_0.pdf.
2. Docker should be installed in the system.
3. Kubernetes should be installed in the system. Instructions on how to do so on a local machine can be found at <https://kubernetes.io/docs/setup/learning-environment/minikube/>.

Now, create a simple directory, as shown in the code snippet below:

```
mkdir flask-kubernetes/app && cd flask-kubernetes/app
```

Next, inside the *flask-kubernetes/app* directory, create a file called *main.py*, as shown in the code snippet below:

```
touch main.py
```

In the newly created *main.py*, paste the following code:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Kubernetes!"

if __name__ == "__main__":
    app.run(host='0.0.0.0')
```

Install Flask in your local using the command below:

```
pip install Flask==0.10.1
```

After installing Flask, run the following command:

```
python app.py
```

This should run the Flask server